

ДЗ: Многоуровневый аналитический слой в ClickHouse

Дедлайн: ...

Prerequisites

- 1. Подключиться к ClickHouse-кластеру (2 шарда × 2 реплики) через ClickHouse Client или DBeaver
- 2. Создать свою базу данных `student<N>`
- 3. Макросы `{shard}` и `{replica}` настроены в конфигурации сервера
- 4. Все DDL выполняются с `ON CLUSTER <cluster>`

Данные

События доставки push-уведомлений доступны в общей таблице `homework_data.push_events_source` (Distributed, 2 шарда).

Поле	Тип	Описание
<code>event_dt</code>	DateTime	timestamp события
<code>event_id</code>	UInt64	ID события (для дедупликации)
<code>user_id</code>	UInt64	ID пользователя (~10K уникальных)
<code>message_id</code>	UInt64	ID сообщения

platform	LowCardinality(String)	ANDROID, IOS, WEB, DESKTOP
status	LowCardinality(String)	delivered, failed, opened
latency_ms	UInt32	задержка доставки, мс (1..5000)

В Задании 1 вы создаёте свою raw-таблицу и копируете данные из `homework_data.push_events_source` через `INSERT INTO ... SELECT ...`. Все дальнейшие агрегаты строятся через Materialized Views автоматически при этом INSERT.

Движки

Слой	Движок	Назначение
Raw	ReplicatedMergeTree + Distributed	Сырые события, репликация внутри шарда, шардирование
Aggregates	AggregatingMergeTree + Distributed	Pre-aggregation: count, uniq, avg
Counters	SummingMergeTree + Distributed	Числовые счётчики: delivered/failed/opened
Dedup	ReplacingMergeTree + Distributed	Дедупликация по event_id

Каждый слой — это пара таблиц: локальная (*_local) + Distributed поверх неё. INSERT всегда в Distributed, SELECT тоже из Distributed. MV пишут в локальные таблицы.

Задание 1 — Raw-слой

Что сделать?

Создать реплицируемую таблицу `push_events_raw_local` на `ReplicatedMergeTree + Distributed`-таблицу `push_events_raw` поверх неё.

Требования:

- Партиционирование по дню (`toYYYYMMDD`)
- TTL 30 дней от `event_dt`
- Ordering key для быстрого поиска по пользователю
- Выбрать правильный sharding key для Distributed (обеспечить равномерное распределение по 2 шардам)

Скопировать данные из `homework_data.push_events_source` в свою Distributed-таблицу через `INSERT INTO ... SELECT ....`

Проверки

```
-- Распределение по шардам:
SELECT hostName(), count() FROM push_events_raw GROUP BY

-- Репликация:
SELECT database, table, is_readonly, absolute_delay, log_
FROM system.replicas WHERE database = currentDatabase();
```

Данные должны быть равномерно распределены по шардам (~1M на шард). `is_readonly = 0`.

Вопрос: что будет, если вставлять данные в локальную таблицу, а не в Distributed?

Задание 2 — Агрегаты (AggregatingMergeTree + MV)

Что сделать?

Создать таблицу `push_events_agg_local` на `AggregatingMergeTree` для pre-aggregation:

- `count, uniq(user_id), avg(latency_ms)` по `(event_date, platform)`
- Колонки типа `AggregateFunction с *State-функциями`

Создать `Distributed push_events_agg` поверх локальной.

Создать `Materialized View`, который при `INSERT` в `raw` обновляет агрегаты. Использовать `countState()`, `uniqState()`, `avgState()`.

Проверки

- Прочитать агрегаты через `countMerge, uniqMerge, avgMerge` с `GROUP BY`
- Сделать `OPTIMIZE TABLE push_events_agg_local FINAL` — проверить, что строки схлопнулись
- Сравнить с прямым запросом из `raw` — результаты должны совпадать

Вопрос: что произойдёт, если `MV` создать **после** вставки данных в `raw`?

Вопрос: что вернёт `avgMerge` без `GROUP BY`? Будет ли это корректным средним?

Задание 3 — Counters (SummingMergeTree + MV)

Что сделать?

Создать таблицу `push_counters_local` на `SummingMergeTree`:

- `delivered_count, failed_count, opened_count` по `(event_date, platform)`
- Включить в схему колонку `user_id UInt64` (намеренно)

Создать `Distributed push_counters` поверх локальной.

Создать `MV` из `raw` через `countIf(status = '...')`.

Проверки

- Вставьте данные в raw, проверьте что counters обновились
- Сделайте `SELECT * FROM push_counters` — посмотрите на `user_id`
- Сделайте `OPTIMIZE TABLE push_counters_local FINAL`
- Сделайте `SELECT * FROM push_counters` снова

Вопрос: что не так с `user_id` после `OPTIMIZE FINAL`? Почему? Как починить (предложите 2 варианта)?

Вопрос: почему `SummingMergeTree` не подходит для расчёта `avg`?

Задание 4 — Дедупликация (`ReplacingMergeTree`)

Что сделать?

Создать таблицу `push_events_dedup_local` на `ReplacingMergeTree(event_dt)`:

- Дедупликация по `event_id` (версия — `event_dt`)

Создать `Distributed push_events_dedup` поверх локальной.

Скопировать данные из `homework_data.push_events_source` в `push_events_dedup`. В источнике присутствуют дубликаты по `event_id` (одинаковый `event_id`, разный `event_dt`).

Проверки

```
SELECT count() FROM push_events_dedup;  
SELECT count() FROM push_events_dedup FINAL;
```

Вопрос: почему результаты разные? Когда дубли реально удаляются? Какая версия (`event_dt`) выигрывает при мёрдже — максимальная или минимальная?

Вопрос: можно ли полагаться на ReplacingMergeTree для строгой дедупликации в моменте?

Задание 5 — Сравнение производительности

Что сделать?

Написать один и тот же запрос (count, uniq, avg по платформам за 7 дней) двумя способами:

1. Из `push_events_raw` (GROUP BY + обычные агрегатные функции)
2. Из `push_events_agg` (Merge-функции)

Сравнить через `system.query_log`:

```
SELECT query_duration_ms, read_rows, read_bytes, query
FROM system.query_log
WHERE event_time > now() - 120
      AND type = 'QueryFinish'
      AND query LIKE '%push_events%'
ORDER BY event_time DESC
LIMIT 10;
```

Вопрос: в каком случае raw может быть быстрее agg?

Критерии оценки

Общие критерии

1. Все DDL и запросы выполнены на кластере (ON CLUSTER)
2. Даны пояснения в markdown-ячейках / комментариях о выбранных решениях
3. Результаты raw и agg запросов совпадают (Задания 2, 5)

На оценку 4

- Выполнены задания 1, 2, 3, 4
- Данные равномерно распределены по шардам
- MV корректно обновляют агрегаты и counters
- Допущено не более 3-х ошибок, которые критически не нарушают логику решения

На оценку 5

- Выполнены все 5 заданий
- Подковырки (SummingMergeTree `user_id`, MV после INSERT, `avgMerge` без GROUP BY) обнаружены, объяснены и починены
- ReplacingMergeTree: разница `count()` vs `FINAL` объяснена
- Сравнение производительности приложено с цифрами из `system.query_log`

Формат сдачи

1. SQL-файл или ноутбук (Jupyter с ClickHouse driver / DBeaver export) со всеми DDL, INSERT и SELECT
2. В комментариях на платформе указать имя БД и приложить результаты проверок:
 - `hostName(), count()` по шардам
 - `system.replicas`
 - `system.query_log` для raw vs agg

Имя БД = `studentX`, где X - ваш номер, присвоенный ранее

ВАЖНО! Все таблицы должны быть созданы на кластере (`ON CLUSTER`), данные вставлены в Distributed-таблицу, репликация активна (`is_readonly = 0`)!

Желаем удачи!